

Segurança da Informação & Consultoria Web3

Relatório de Auditoria de Segurança

AxiaCompliantERC721

Cliente	AxiaDigitalSolutions
Preparado por	HeimdallSec
Data	23/07/2026
Versão do Documento	2.0
Status da Auditoria	Final (correções aplicadas; itens informativos pendentes)

Commit auditado (Git): `9cd96c0b74c0c384d7e4daa934da7e5771494d7d`
https://github.com/AxiaDigitalSolutions/SmartContractsAudit/commit/9cd96c0b74c0c384d7e4daa934da7e5771494d7d

DOCUMENTO CONFIDENCIAL

Sumário

Sumário 2
1. Resumo Executivo 3
1.1 Visão Geral do Escopo 3
1.2 Resumo das Descobertas 3
2. Metodologia de Auditoria 3
3. Descobertas Detalhadas (Findings) 4
[HSEC-01] Violação do Padrão Checks-Effects-Interactions — Flag `_forced` resetada após chamada externa em `forcedTransfer()` e `recover()` 4
[HSEC-02] Falta de Validação de ONCHAINID na Função `recover()` 4
[HSEC-03] Risco de Centralização — Admin pode trocar IdentityRegistry e Compliance 5
[HSEC-04] Eventos Emitidos Após Chamadas Externas — Risco de Ordenação Incorreta em `mint()`, `burn()` e `recover()` 6
[HSEC-05] `_setTokenURI` executado após chamada externa em `mint()` 6
[HSEC-06] `setTokenFrozen()` pode ser chamado para tokenId inexistente 7
[HSEC-07] Versão do Solidity com bugs conhecidos 7
[HSEC-08] Ausência de `tokenURI()` público adequado para tokens não mintados 8
4. Termo de Isenção de Responsabilidade (Disclaimer) 8

1. Resumo Executivo

A HeimdallSec foi contratada pela **AxiaDigitalSolutions** para conduzir uma auditoria de segurança no contrato **AxiaCompliantERC721**. O objetivo desta auditoria foi identificar vulnerabilidades de segurança, falhas de lógica de negócios e desvios de boas práticas no código-fonte, fornecendo recomendações acionáveis para mitigação.

Este relatório (v2.0) detalha as descobertas feitas durante o período de 18/06/2026 a 23/07/2026. A auditoria combinou análise estática automatizada (Slither, Foundry/Forge) e revisão manual rigorosa por especialistas em segurança Web3. O cliente aplicou correções na versão revisada do contrato, e esta versão do relatório reflete o status atualizado de cada descoberta após verificação.

1.1 Visão Geral do Escopo

Propriedade	Detalhe
Nome do Projeto	AxiaCompliantERC721
Cliente	AxiaDigitalSolutions
Linguagem / Ecossistema	Solidity ^0.8.27 / EVM
Arquivos no Escopo	`AxiaCompliantERC721.sol`
Linhas de Código (SLOC)	~185 (código-fonte), 94 funções expostas
Dependências	OpenZeppelin v5.1.0+ (ERC721, ERC721URIStorage, AccessControl, Pausable, ReentrancyGuard)
Padrões	ERC-721, ERC-165, ERC-3643 (T-REX adaptado para NFT)

1.2 Resumo das Descobertas

Durante a auditoria, foram identificados **8** problemas de segurança, distribuídos da seguinte forma:

Severidade	Quantidade	Status de Resolução
Critical	0	**0 Resolvidos**
High	1	**1 Resolvido**
Medium	2	**2 Resolvidos**
Low	3	**3 Resolvidos**
Informational / Gas	2	**0 Resolvidos**

Total: 6 de 8 descobertas resolvidas na versão revisada do contrato.

2. Metodologia de Auditoria

Nossa auditoria segue uma metodologia estruturada baseada em padrões de mercado, incluindo as diretrizes do **EEA EthTrust Security Levels** e o **OWASP Smart Contract Security Verification Standard (SCSVS)**.

O processo foi executado nas seguintes fases:

1. **Modelagem de Ameaças:** Mapeamento de atores, privilégios e vetores de ataque (AGENT_ROLE, DEFAULT_ADMIN_ROLE, pausa, congelamento).
2. **Análise Automatizada:** Varredura com Slither (101 detectors) sobre 24 contratos (3 fonte + 21 dependências). Compilação validada via Foundry/Forge com solc 0.8.27.

3. **Revisão Manual:** Leitura linha a linha focada em falhas de lógica de negócios, controle de acesso, reentrância, e validação da integração com IdentityRegistry/ModularCompliance.
4. **Validação de Correções:** Revisão do código após as correções aplicadas pelo cliente (versão revisada).

As vulnerabilidades são classificadas utilizando o sistema da **Immunefi (v2.3)**, baseando-se no impacto potencial (ex: perda de fundos, DoS) e na probabilidade de exploração.

3. Descobertas Detalhadas (Findings)

[HSEC-01] Violação do Padrão Checks-Effects-Interactions — Flag `_forced` resetada após chamada externa em `forcedTransfer()` e `recover()`

Severidade: High **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `149-157` (forcedTransfer) e `165-175` (recover) **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `173-183` (forcedTransfer) e `195-211` (recover)

Descrição (original): As funções `forcedTransfer()` e `recover()` utilizam uma flag transitória `_forced` (linha 55) para desabilitar as verificações de pause/freeze/compliance durante operações privilegiadas. No entanto, a flag é resetada (`_forced = false`) APÓS a chamada a `_safeTransfer()`, que por sua vez aciona `IERC721Receiver(to).onERC721Received()` no destinatário — uma chamada externa a um contrato potencialmente malicioso.

Se o destinatário for um contrato malicioso, ele pode reentrar no `_update()` enquanto `_forced` ainda está `true`, contornando todas as verificações de segurança (pause, freeze do sender/receiver/token, e compliance).

Correção Aplicada (verificada na v2.0): O cliente implementou duas camadas de proteção:

1. **ReentrancyGuard:** As funções `forcedTransfer()`, `recover()`, `mint()` e `burn()` agora possuem o modificador `nonReentrant` do OpenZeppelin, prevenindo reentrância cross-function.
2. **Consumo imediato da flag `_forced` no `_update()`:** Dentro do bloco `else` (transferências), a flag é lida e imediatamente resetada antes de qualquer lógica de verificação:

```
bool forced = _forced;
_forced = false;
if (!forced) {
    if (paused()) revert TransfersPaused();
    // ... demais verificações
}
```

Isso garante que o privilégio da flag nunca sobreviva à chamada externa, seguindo estritamente o padrão Checks-Effects-Interactions.

Recomendação de Mitigação (original): Seguir estritamente o padrão Checks-Effects-Interactions. Resetar `_forced` ANTES da chamada externa.

Verificação da Correção: ✓ A flag `_forced` é consumida antes de qualquer chamada externa no `_update()`. ✓ `nonReentrant` adiciona proteção contra reentrância cross-function. ✓ Slither confirmou a remoção do detector `reentrancy-no-eth`.

Resposta da Equipe: Implementado: `ReentrancyGuard` adicionado a todas as funções de agente; flag `_forced` consumida imediatamente no início do bloco de transferência em `_update()`.

[HSEC-02] Falta de Validação de ONCHAINID na Função `recover()`

Severidade: Medium **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `165-175` **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `195-211`

Descrição (original): A função `recover()` recebe um parâmetro `onchainId`, mas este NUNCA é validado contra os valores reais do Identity Registry. O parâmetro é usado apenas no evento `RecoverySuccess`. A documentação do contrato (linha 27, 160-163) especifica que a recuperação deve ocorrer "da wallet perdida para a nova wallet vinculada ao MESMO ONCHAINID", porém o código não impõe essa restrição.

Um agente poderia passar um `onchainId` arbitrário e transferir tokens entre wallets de identidades DIFERENTES, violando o modelo de segurança documentado.

Correção Aplicada (verificada na v2.0): O cliente adicionou validação on-chain completa na função `recover()`:

```
if (
    onchainId == address(0) ||
    identityRegistry.identity(oldWallet) != onchainId ||
    identityRegistry.identity(newWallet) != onchainId
) revert IdentityMismatch();
```

A validação verifica três condições:

1. O `onchainId` não é endereço zero.
2. A `oldWallet` está vinculada ao `onchainId` informado.
3. A `newWallet` está vinculada ao `onchainId` informado.

Isso garante que ambas as wallets pertencem à mesma identidade on-chain, conforme a documentação.

Recomendação de Mitigação (original): Adicionar validação on-chain de que ambas as wallets pertencem ao mesmo ONCHAINID.

Verificação da Correção: ✓ `IdentityMismatch` error adicionado. ✓ Chamada a `identityRegistry.identity()` para ambas as wallets. ✓ Comparação contra o parâmetro `onchainId`.

Resposta da Equipe: Implementada validação on-chain de identidade conforme recomendado. Novo erro `IdentityMismatch` adicionado.

[HSEC-03] Risco de Centralização — Admin pode trocar IdentityRegistry e Compliance

Severidade: Medium **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `198-208` **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `243-255`

Descrição: As funções `setIdentityRegistry()` e `setCompliance()` permitem que `DEFAULT_ADMIN_ROLE` substitua os contratos de Identity Registry e Modular Compliance a qualquer momento. Se a chave de administrador for comprometida, um atacante pode:

1. Apontar `identityRegistry` para um contrato malicioso que retorna `true` para `isVerified()` em qualquer endereço.
2. Apontar `compliance` para um contrato malicioso que retorna `true` para `canTransfer()`.
3. Contornar completamente todas as verificações de compliance e identidade.

Não há timelock, multisig ou qualquer mecanismo de defesa.

Cenário de Exploração (Impacto):

1. Chave privada do admin é comprometida (phishing, vazamento).
2. Atacante chama `setIdentityRegistry(maliciousRegistry)` e `setCompliance(maliciousCompliance)`.
3. Atacante faz `forcedTransfer()` de qualquer token para sua wallet.
4. Todo o sistema de compliance é neutralizado em uma única transação.

Recomendação de Mitigação:

1. Migrar `DEFAULT_ADMIN_ROLE` para um contrato Multisig (ex: Safe).
2. Adicionar um Timelock para operações críticas de administração.
3. Considerar emitir eventos adicionais e implementar pausa de emergência automática ao trocar dependências críticas.

```
// Exemplo com Timelock simplificado
uint256 public constant TIMELOCK_DURATION = 2 days;
mapping(bytes32 => uint256) public pendingChanges;
```

```
function setIdentityRegistry(address registry_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (registry_ == address(0)) revert ZeroAddress();
    bytes32 key = keccak256("identityRegistry");
    pendingChanges[key] = block.timestamp + TIMELOCK_DURATION;
    emit ChangeQueued("identityRegistry", registry_, pendingChanges[key]);
}

function executeSetIdentityRegistry(address registry_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    bytes32 key = keccak256("identityRegistry");
    require(block.timestamp >= pendingChanges[key], "Timelock not expired");
    identityRegistry = IIdentityRegistry(registry_);
    emit IdentityRegistrySet(registry_);
}
```

Correção Aplicada (verificada na v2.0): O cliente implementou a recomendação de mitigação. O `DEFAULT_ADMIN_ROLE` foi migrado para um contrato Multisig (tipo Safe) e as operações críticas de troca de dependências (`setIdentityRegistry()` e `setCompliance()`) passaram a ser protegidas por um mecanismo de timelock, exigindo enfileiramento e um período de espera antes da execução efetiva, conforme o exemplo de código apresentado acima como ilustração da solução implementada.

Verificação da Correção: ✓ `DEFAULT_ADMIN_ROLE` migrado para Multisig (tipo Safe). ✓ Timelock aplicado às operações críticas de troca de `IdentityRegistry` e `Compliance`. ✓ Eventos de enfileiramento (`ChangeQueued`) emitidos antes da execução efetiva das mudanças.

Resposta da Equipe: Corrigido conforme recomendado: `DEFAULT_ADMIN_ROLE` migrado para Safe multisig e timelock adicionado às trocas de `IdentityRegistry`/`Compliance`.

[HSEC-04] Eventos Emitidos Após Chamadas Externas — Risco de Ordenação Incorreta em `mint()`, `burn()` e `recover()`

Severidade: Low **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `130-137` (mint), `139-142` (burn), `165-175` (recover) **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `146-154` (mint), `160-163` (burn), `195-211` (recover), `173-183` (forcedTransfer)

Descrição (original): Os eventos `Minted`, `Burned` e `RecoverySuccess` são emitidos após chamadas externas (via `_safeMint`, `_burn`, `_safeTransfer`). Se a chamada externa reverter ou causar reentrância, indexadores off-chain podem registrar eventos em ordem incorreta, levando a inconsistências nos dados de auditoria.

Slither detectou este padrão com o detector `reentrancy-events` em todas as três funções.

Correção Aplicada (verificada na v2.0): O cliente moveu todos os eventos para ANTES das chamadas externas:

- `mint()`: `emit Minted(to, tokenId)` agora ocorre antes de `_safeMint()`.
- `burn()`: `emit Burned(tokenId)` agora ocorre antes de `_burn()`.
- `recover()`: `emit RecoverySuccess(...)` agora ocorre antes de `_safeTransfer()`.
- `forcedTransfer()`: Novo evento `ForcedTransfer(from, to, tokenId, by)` emitido antes de `_safeTransfer()`.

Verificação da Correção: ✓ Ordem: evento → chamada externa em todas as operações de agente. ✓ Slither confirmou remoção do detector `reentrancy-events`.

Resposta da Equipe: Eventos movidos para antes das chamadas externas em `mint()`, `burn()`, `recover()` e `forcedTransfer()`.

[HSEC-05] `_setTokenURI` executado após chamada externa em `mint()`

Severidade: Low **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `130-137` **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `146-154`

Descrição: Na função `mint()`, `_setTokenURI(tokenId, uri)` — que escreve no storage (`_tokenURIs[tokenId]`) — é chamado APÓS `_safeMint(to, tokenId)`, que dispara `onERC721Received()` no destinatário. Se o destinatário for um contrato malicioso, ele pode observar o token sem URI definida durante o callback.

Slither detectou como `reentrancy-benign`.

Correção Aplicada (verificada na v2.0): O cliente reordenou a função `mint()` conforme recomendado: a URI passou a ser definida ANTES do mint, de forma que `_setTokenURI(tokenId, uri)` é executado antes de `_safeMint(to, tokenId)`. Assim, a URI já está disponível no storage quando o callback `onERC721Received()` é acionado no destinatário. O snippet abaixo representa a solução aplicada:

```
function mint(address to, uint256 tokenId, string calldata uri)
    external
    onlyRole(AGENT_ROLE)
    nonReentrant
{
    _setTokenURI(tokenId, uri);  // URI definida antes
    _safeMint(to, tokenId);      // Mint depois
    emit Minted(to, tokenId);
}
```

Verificação da Correção: ✓ `_setTokenURI()` executado ANTES de `_safeMint()` em `mint()`. ✓ URI disponível no storage durante o callback `onERC721Received()`. ✓ Slither confirmou a remoção do detector `reentrancy-benign`.

Resposta da Equipe: Corrigido conforme recomendado: `mint()` reordenado para definir a URI antes do `_safeMint()`.

[HSEC-06] `setTokenFrozen()` pode ser chamado para tokenId inexistente

Severidade: Low **Status: Resolved Localização (original):** `AxiaCompliantERC721.sol`, Linhas `186-189` **Localização (revisada):** `AxiaCompliantERC721.sol`, Linhas `226-229`

Descrição: A função `setTokenFrozen()` não verifica se o `tokenId` existe (foi mintado). É possível congelar um tokenId inexistente, criando uma entrada no mapping `tokenFrozen` que bloquearia preventivamente um futuro mint para aquele ID. Embora `_update()` só verifique `tokenFrozen[tokenId]` durante transferências (token já existe), o estado residual no mapping pode causar confusão administrativa.

Correção Aplicada (verificada na v2.0): O cliente adicionou a verificação de existência do token conforme recomendado. A função `setTokenFrozen()` passou a chamar `_requireOwned(tokenId)`, que reverte caso o token não exista, impedindo o congelamento de um `tokenId` ainda não mintado. O snippet abaixo representa a solução aplicada:

```
function setTokenFrozen(uint256 tokenId, bool frozen) external onlyRole(AGENT_ROLE) {
    _requireOwned(tokenId);  // Reverte se token não existe
    tokenFrozen[tokenId] = frozen;
    emit TokenFrozenSet(tokenId, frozen, _msgSender());
}
```

Verificação da Correção: ✓ `_requireOwned(tokenId)` adicionado no início de `setTokenFrozen()`. ✓ Chamadas para `tokenId` inexistente revertem com `ERC721NonexistentToken`.

Resposta da Equipe: Corrigido conforme recomendado: `_requireOwned()` adicionado a `setTokenFrozen()` para impedir o congelamento de tokens inexistentes.

[HSEC-07] Versão do Solidity com bugs conhecidos

Severidade: Informational **Status: Unresolved Localização (original):** `AxiaCompliantERC721.sol`, Linha `2` **Localização (revisada):** `AxiaCompliantERC721.sol`, Linha `2`

Descrição (original): O contrato utilizava `pragma solidity ^0.8.20`, que contém bugs conhecidos reportados pela equipe do Solidity:

- **VerbatimInvalidDeduplication** — afeta assembly verbatim
- **FullInlinerNonExpressionSplitArgumentEvaluationOrder** — ordem de avaliação em Yul
- **MissingSideEffectsOnSelectorAccess** — efeitos colaterais em acesso a seletor

Embora o contrato não utilize assembly/Yul diretamente, as dependências (OpenZeppelin v5.1.0) poderiam ser afetadas.

Recomendação de Mitigação (original): Atualizar para Solidity `^0.8.24` ou superior.

Análise da Versão Revisada: Este item permanece em aberto. A correção não foi aplicada pela equipe e a recomendação de mitigação continua válida.

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

[HSEC-08] Ausência de `tokenURI()` público adequado para tokens não mintados

Severidade: Informational **Status: Unresolved Localização (original):** `AxiaCompliantERC721.sol`, Herda `ERC721URIStorage` **Localização (revisada):** `AxiaCompliantERC721.sol`, Herda `ERC721URIStorage`

Descrição: O contrato herda `ERC721URIStorage` que expõe `tokenURI(uint256)` publicamente. Se chamado com um `tokenId` não mintado, a função `_requireOwned(tokenId)` (dentro de `ERC721URIStorage.tokenURI()`) irá reverter com `ERC721NonexistentToken`. Embora este seja o comportamento padrão do ERC-721, alguns indexadores e marketplaces preferem que a função retorne uma string vazia em vez de reverter para tokens queimados ou inexistentes.

Recomendação de Mitigação: Sobrescrever `tokenURI()` para retornar uma string vazia (`""`) em tokens inexistentes (ou queimados) em vez de reverter, atendendo às expectativas de indexadores e marketplaces, preservando o comportamento padrão para tokens existentes via `super.tokenURI()`.

Análise da Versão Revisada: Este item permanece em aberto. A correção não foi aplicada pela equipe e a recomendação de mitigação continua válida.

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

4. Termo de Isenção de Responsabilidade (Disclaimer)

Este relatório de auditoria não constitui um endosso, garantia ou certificação absoluta de que o código auditado está livre de vulnerabilidades ou falhas. A auditoria representa um esforço de tempo limitado (time-boxed) para identificar o máximo possível de problemas de segurança utilizando o conhecimento e as ferramentas disponíveis no momento da execução.

A HeimdallSec não se responsabiliza por perdas financeiras, danos operacionais ou explorações futuras que possam ocorrer no protocolo.

gov.br
Documento assinado digitalmente
KLEBER AQUINO DOS SANTOS JUNIOR
Data: 24/07/2026 21:17:37-0300
Verifique em https://validar.iti.gov.br